

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/359045213>

A New Algorithm for Data Migration from a Relational to a NoSQL Oriented Column Database

Chapter · March 2022

DOI: 10.1007/978-3-030-94191-8_64

CITATIONS

3

READS

110

3 authors, including:



Ahmed Dourhri

Université Chouaib Doukkali

4 PUBLICATIONS 4 CITATIONS

SEE PROFILE



Mohamed Hanine

Chouaib Doukkali University

49 PUBLICATIONS 621 CITATIONS

SEE PROFILE

A new algorithm for data migration from a relational to a NoSQL oriented column database

Ahmed DOURHRI¹, Mohamed HANINE², and Hassan OUAHMANE

LTI laboratory, ENSA, Chouaib Doukkali University, El Jadida
dourhriahmed@gmail.com, m.hanine.ensaj@gmail.com,
hassan.ouahmane@yahoo.fr

Abstract. Currently, the rapid progression of technology has driven systems to manage a huge amount of data emanating from various sources such as social networks, event logs, websites surfing, and many other sources. Since 2008, many companies have helped develop a set of technologies to replace the classic systems called RDBMS (Relational Database Management System). A new need has just been created, namely, replacing the old databases mainly relational to their NoSQL (Not Only SQL) counterpart. This work aims to present a new methodology for preparing NoSQL oriented-column databases from relational databases. To illustrate this methodology, we developed software using PostgreSQL as an RDBMS and Cassandra as a NoSQL oriented-column database. Despite it being hard engineering work, the obtained results show that our proposal can effectively solve the problem posed by this article.

Keywords: Data migration, Reverse engineering, PostgreSQL, RDBMS, NoSQL, Cassandra

1 Introduction

During the past years, we have seen that systems managing a large volume of data are gradually moving towards NoSQL databases offering several advantages (flexibility of data schemas, short response time, etc.). A need has arisen for the old systems (mainly relational) which want to convert to the NoSQL environment: this is the subject of this research paper. To illustrate this conversion, we have chosen PostgreSQL as RDBMS (Relational database management system) and CASSANDRA as NoSQL oriented-column DBMS. The process of this conversion called migration consists of 3 steps [1]: i) extract the data from the data source (for us, a PostgreSQL database), ii) transform (combine the collected data with building an adequate structure), iii) load the structure and data in the target database (Cassandra). The main reasons that led us to choose Cassandra as a target database are its various advantages:

- Very fast to handle a large volume of data;
- Flexible data schemas;

¹ ORCID : 0000-0001-7246-3707

² ORCID : 0000-0001-5981-2511

- Possibility to put in cluster several Cassandra servers;
- Data replication.

Before presenting the steps of such migration, we will focus on relational databases using the famous SQL language for querying and NoSQL databases. Each NoSQL system has its language.

Relational database: a relational database is a database that relies on the relational model based on set theory and having been invented by mathematician Edgar Codd in 1970. This model consists of dividing the data into tables formed rows and columns possibly interconnected via foreign keys [1]. The systems managing such databases are called relational database management systems (RDBMS). They use for data manipulating the SQL the abbreviation of Structured Query Language. The strong point of these systems is the optimization of the memory space necessary for data management (to the detriment of speed).

NoSQL database: NoSQL systems come to speed up the querying of databases by offering the possibility of creating flexible schemes and guarantee scalability if necessary. Several models are available for these systems [2].

- The key-value model: data is stored in the form of a dictionary. This type allows keeping high performance in reading and, or writing. Several products, opting for this model, exist as Voldemort, Redis, Riak...
- Document model: this model extends the key values model in the sense that a database is a set of documents represented in a standard format (JSON: Javascript Object Notation, BSON, etc.). The advantage of this model is the recovery of a set of hierarchically structured information from a single key. Among the well-known implementations of this model, we cite MongoDB and CouchDB.
- Graph model: graph-oriented databases are those that store records in nodes and relationships between records by edges. This type of database is very efficient where there are a huge amount of data strongly connected. Neo4j, HyperGraphDB are examples of graph-oriented DBMS.

Column-oriented model: the notion of a table in this model, is different from the notion of a table in a relational database. In this model, a table can be viewed as a set of partitions that contain rows with a similar structure. Each partition in a table has a unique partition key and each row in a partition may optionally have a unique clustering key. Both keys can be simple, i.e. one column or composite, i.e. Multiple columns. Combining a partition key and a clustering key uniquely identifies a row in a table and is called a primary key. To illustrate some of these notions, Figure 1 shows tables with CQL (Cassandra Query language) definitions and sample rows. In Figure 1(a), the Account table contains single-row partitions. Its primary key consists of one column num (number of an account) that is also a simple partition key. This table is shown to have three single-row partitions. In Figure 1(b), the detailed_account table contains multi-row partitions. Its primary key consists of partition key (num) and clustering key op_num. This table is shown to have three partitions, each one containing multiple rows. For any given partition, its rows are ordered by op_num in ascending order. Figure 2 shows the model structure of Cassandra as an example of a

column-oriented model, and Table 1 shows some SQL concepts and their correspondences in Cassandra.

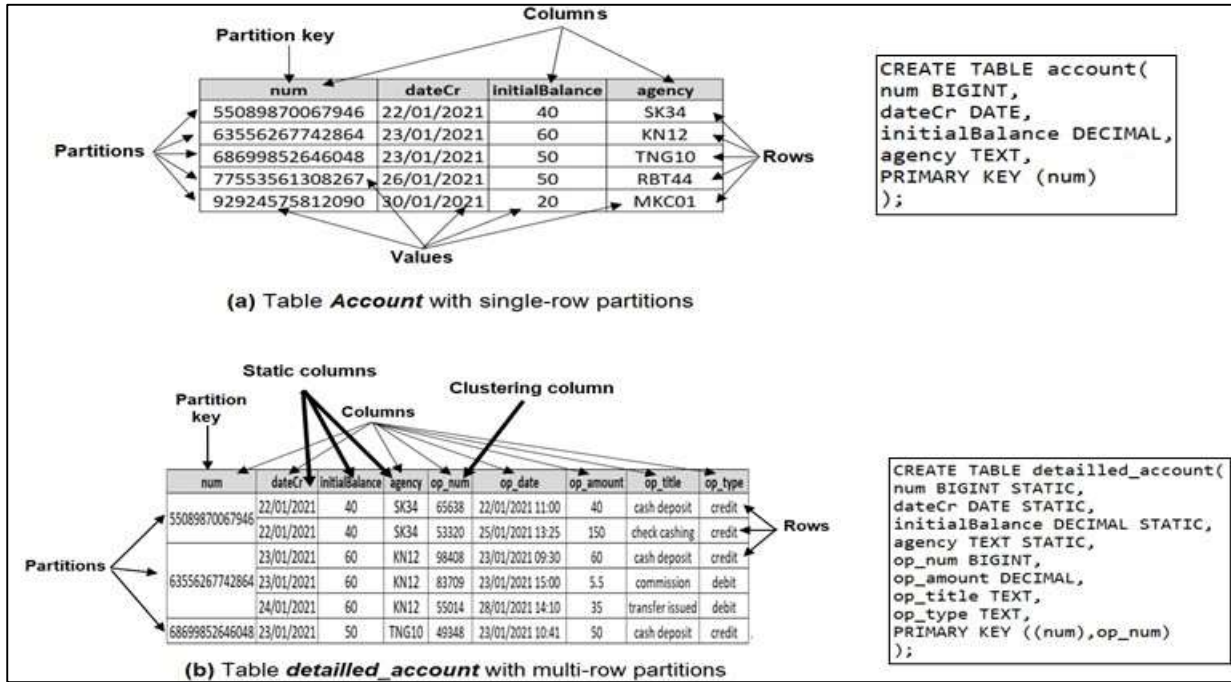


Figure1: Sample tables in Cassandra [2][3]

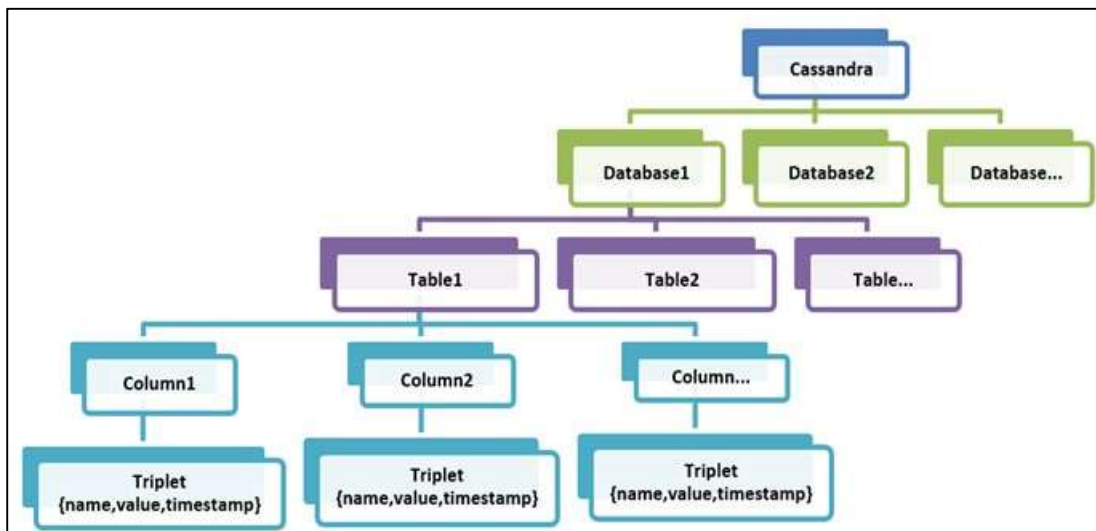


Figure2: Model structure of Cassandra

SQL terminology	Cassandra Terminology
Database	Database (or keyspace)
Table	Table (or column family)
Row	Row
Column	Column
Primary key	Primary key
join of tables	Nesting with collections (by the use of sets, maps, or lists)
CREATE TABLE Users (user_id int, name text, surname text, city text, PRIMARY KEY (user_id));	CREATE TABLE Users (user_id int, name text, surname text, city text, PRIMARY KEY (user_id));
INSERT INTO Users (user_id, name, surname, city) VALUES (1745, 'ayoub', 'mohamed', 'casa');	INSERT INTO Users (user_id, name, surname, city) VALUES (1745, 'ayoub', 'mohamed', 'casa');
CREATE INDEX ON Users (surname);	CREATE INDEX ON Users (surname);
DROP INDEX Users_surname_idx ;	DROP INDEX users_surname_idx ;
ALTER TABLE Users ADD phone text;	ALTER TABLE Users ADD telephone text;
UPDATE Users SET phone = '21212121' WHERE user_id=1745;	UPDATE Users SET phone = '21212121' WHERE user_id=1745;
DELETE FROM Users WHERE user_id= '176';	DELETE FROM Users WHERE user_id= '176';
SELECT * FROM Users WHERE surname = 'mohamed';	SELECT * FROM Users WHERE surname = 'mohamed';
DELETE FROM users;	TRUNCATE users;

Table1: Basic SQL vs Cassandra Syntax (adapted from [1,3])

2 PROPOSED METHODOLOGY

The main aim of this study is to present a methodology for facilitating the migration from an RDBMS (PostgreSQL) to NoSQL (Cassandra). The main steps of the methodology are visually displayed in Figure 3, which, as highlighted in black circles, explain how the methodology works. To present each step, we shall use an example of a database representing the accounts opened in a bank in Figure 4.

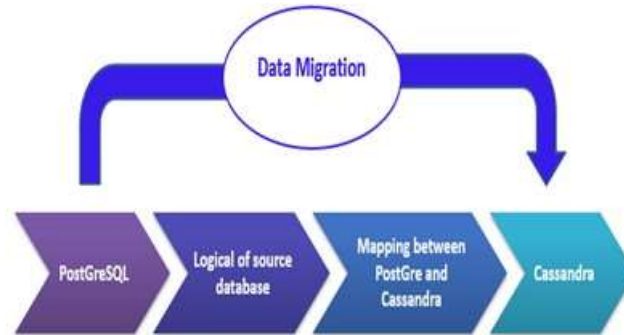


Figure3: Data migration methodology

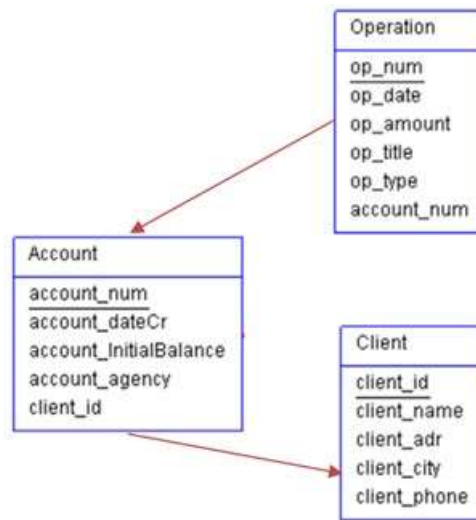


Figure4: Relational database logical structure

2.1 Loading the Logical Structure of the Source Database

Initially, we will have to specify the source of the relational database. Then we connect to this source to retrieve all information on type and version. We will need to provide all the information on the target NoSQL database. Besides, in this step, we get a representation of the relational model of the database. For this, we will need to have the names of tables, their attributes, and relationships. Relationship information can be deduced from primary keys and foreign keys from each table.

2.2 Mapping Between Relational Model and Cassandra

This step is dedicated to defining the mapping between the PostgreSQL relational model and the oriented model columns of Cassandra. It consists of defining the attributes of the relational model that will be linked in the target model. Loading tables from the database and their attributes can be implemented by more than one method. During

the migration of the relational model, the system will propose a set of suitable tables to migrate into the database target.

The main concern of those coming from a relational background is the absence of joins in NoSQL databases. As shown below, the column-oriented model makes joins redundant in several cases. In Figure 5, the relational database uses the attribute "client_id" to join the "Account" table with the table "Client". The attribute "account_num" is used to join the "Account" table with the table "Operation" to give the application the possibility to retrieve the operations of an account.

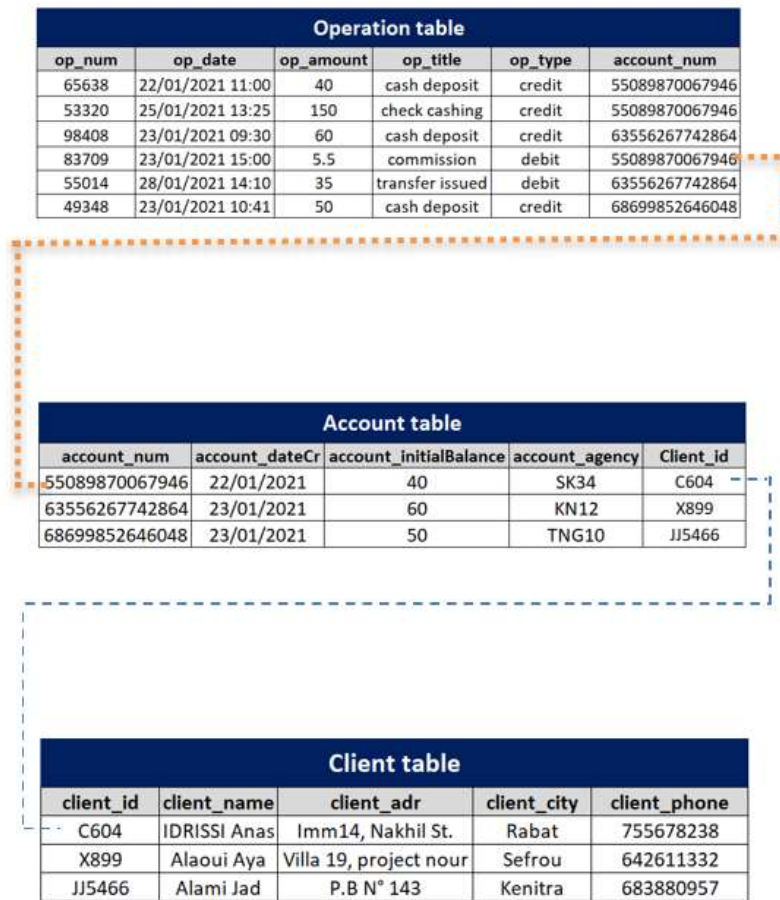


Figure 5: Relational schema (Account-Client-Operation)

After we used the columns model, collections are used to aggregate data with a single data structure. Attributes that have been traditionally divided into separate, well-structured tables can now be stored in a single table, eliminating the need to join tables separated when the application needs to obtain complete records. With Cassandra, we will create a table "detailed_account" that contains all the details of the account (Figure 6):

Detailed_Account		
account_num	int	K
account_dateCr	date	S
account_initialBalance	decimal	S
account_agency	text	S
client_id	text	S
client_name	text	S
client_adr	text	S
client_city	text	S
client_phone	int	S
op_num	int	C
op_date	timestamp	
op_amount	decimal	
op_title	text	
op_type	boolean	

Figure6: Column-oriented schema

To transform the relational schema into a column-oriented schema, we carry out reverse engineering to obtain the conceptual schema. Then we generate the column-oriented schema according to the associations linking entities of the conceptual schema.

Reverse engineering

Reverse engineering is the activity of studying an object to determine how it works. The objective can be, for example, to create another object with identical functionalities of the starting object without patent infringement.

In the case of the database, reverse engineering consists of finding the starting model from the physical model. We go from the physical model to the logical model, and we deduce the conceptual model from it. Rules [4] must be applied to obtain the conceptual schema from the logical schema:

- **Rule1:** each table whose primary key does not contain a foreign key becomes an entity whose identifier is the primary key of the table and whose attributes are the columns of the table which are not foreign keys.
- **Rule2:** each table whose primary key is composed exclusively of foreign keys which reference several primary keys, becomes an association around which all the maximum cardinalities are equal to *, that is to say, either a binary association of type many-to-many or a ternary association or more (the other columns which are not foreign keys of the table become attributes of the association).
- **Rule3:** the remaining foreign key columns become type one-to-many binary associations if there is no uniqueness constraint or type one-to-one if there is a unique constraint (a name must be found for this association)
- **Rule4:** the minimum cardinality is 1 for foreign keys that are part of a primary key or that have a constraint (not null). Otherwise, it is 0.

The transition from conceptual model to column-oriented logic model [5,6]

a) 1st configuration: one-to-many association

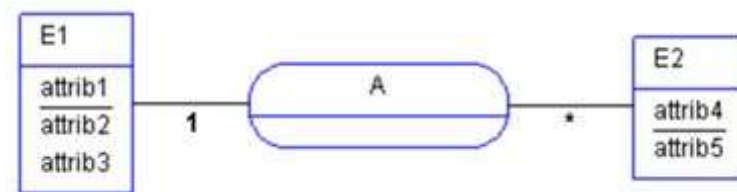


Figure7: one-to-many relationship

Example for illustration



Figure8: Example of a many-to-many relationship

Several options are available to translate this conceptual schema into a column-oriented logical schema such as Cassandra:

-Option n°1: a table named E2sByE1 will be created: entity E2 named as the migrated entity will migrate into entity E1 named as the receiving entity. The E2sByE1 table will have a partition key (key of E1) and a clustering key (key of E2). The CQL code would therefore be:

```

CREATE TABLE E2sByE1(
    attrib1...,
    attrib2...,
    attrib3...,
    attrib4...,
    attrib5...,
    primary key((attrib1),attrib4)
);
  
```

illustration with the example above

```

CREATE TABLE OrdersByClient(
    clt_code ... ,
    clt_name ...,
    clt_phone ...,
    ord_num ...,
    ord_date ...,
    primary key((clt_code),ord_num)
);
  
```

-Option n°2: a table named E2sByE1 will be created: entity E2 named as the migrated entity will migrate into entity E1 named as the receiving entity). The E2sByE1 table will have the attribute attribute1 as its primary key. Occurrences of E2 will be inserted as a list of tuples. The CQL code would therefore be:

```
CREATE TABLE E2sByE1(
  attrib1 ...,
  attrib2 ...,
  attrib3 ...,
  liste_of_E2s list<frozen<tuple<typeA4,typeA5>>>>,
  primary key(attrib1)
);
```

illustration with the example above

we assume that ord_num is a long integer

```
CREATE TABLE OrdersByClient(
  clt_code ... ,
  clt_nom ...,
  clt_phone ...,
  ord_num ...,
  ord_date ...,
  liste_of_orders list<frozen<tuple<bigint, date>>>>,
  primary key(clt_code)
);
```

-Option n°3: as in relational, it is the E1 table that will migrate into the E2 table. Each occurrence of E2 is associated with an occurrence of E1. The latter will be represented as a tuple. The CQL code would therefore be:

```
CREATE TABLE DetailedE1(
  attrib4 ...,
  attrib5 ...,
  E1 tuple<typeAttrib1, typeAttrib2, typeAttrib3>,
  primary key(attrib4)
);
```

illustration with the example above

```
CREATE TABLE DetailedOrder(
  ord_num ... ,
  ord_date ...,
  Client tuple<bigint,text,text>,
  primary key(ord_num)
);
```

It is assumed that the types of attributes of the Client entity are respectively "long integer", "string" and "string".

b) 2nd configuration: many-to-many association

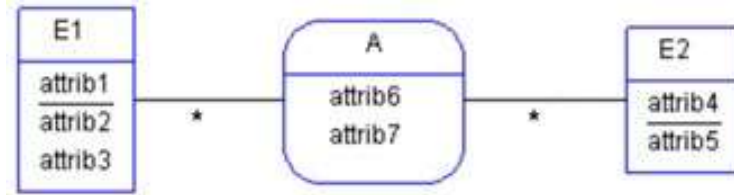


Figure9: Many-to-many relationship



Figure10: Example of a many-to-many relationship

Several options are available to translate this conceptual schema into a column-oriented logical schema.

-Option n°1: a table (column-family) named E2sByE1 will be created: entity E2 named as the migrated entity will migrate into entity E1 named as the receiving entity. The E2sByE1 table will have a partition key (key of E1) and a clustering key (the key of E2). The CQL code would therefore be:

```
CREATE TABLE E2sByE1(
  attrib1...,
  attrib2...,
  attrib3...,
  attrib4...,
  attrib5...,
  attrib6 ...,
  attrib7 ...,
  primary key((attrib1),attrib4)
);
```

illustration with the example above

```
CREATE TABLE OrdersByProduct(
  prod_ref ...,
  prod_name ...,
  prod_unitPrice ...,
  ord_num ...,
  ord_date ...,
  quantity ...,
  primary key((prod_ref),ord_num)
);
```

-Option n°2: a table (column-family) named E2sByE1 will be created: entity E2 named as the migrated entity will migrate into entity E1 named as the receiving entity.

The E2sByE1 table will have the attribute attribute1 as its primary key. Occurrences of E2 will be inserted as a list of tuples. The CQL code would therefore be:

```
CREATE TABLE E2sByE1(
  attrib1 ...,
  attrib2 ...,
  attrib3 ...,
  liste_of_E2s list<frozen<tuple<typeA4, typeA5,typeA6,typeA7>>>,
  primary key(attrib1)
);
```

illustration with the example above

we assume that ord_num and quantity are integers.

```
CREATE TABLE OrdersByProduct(
  prod_ref ... ,
  prod_name...,
  prod_unitPrice ...,
  liste_of_orders list<frozen<tuple<int,date,int>>>,
  primary key(prod_ref)
);
```

-Option n°3: a table (column-family) named E1sByE2 will be created: entity E1 named as the migrated entity will migrate into entity E2 named as the receiving entity. The E1sByE2 table will have a partition key (key of E2) and a clustering key (the key of E1). The CQL code would therefore be:

```
CREATE TABLE E1sByE2(
  attrib4 ...,
  attrib5 ...,
  attrib1 ...,
  attrib2 ...,
  attrib3 ...,
  attrib6 ...,
  attrib7 ...,
  primary key((attrib4),attrib1)
);
```

illustration with the example above

```
CREATE TABLE ProductsByOrder(
  ord_num ...,
  ord_date ...,
  prod_ref ... ,
  prod_name ...,
  prod_unitPrice ...,
  quantity ...,
  primary key((ord_num),prod_ref)
);
```

-Option n°4: a table (column-family) named **E1sByE2** will be created: entity E1 named as the migrated entity will migrate into entity E2 named as the receiving entity.

The E1sByE2 table will have the attribute attribute4 as its primary key. Occurrences of E1 will be inserted as a list of tuples. The CQL code would therefore be:

```
CREATE TABLE E1sByE2(
  attrib4 ...,
  attrib5 ...,
  liste_of_E1s list<frozen<tuple<typeA1, typeA2,typeA3,typeA6,typeA7>>>,
  primary key(attrib4)
);
```

illustration with the example above

we assume that prod_ref and quantity are integer and prod_name is a string. prod_unit-Price is a float.

```
CREATE TABLE ProductsByOrder(
  ord_num ...,
  ord_date ...,
  liste_of_products list<frozen<tuple<int,string,int,float>>>,
  primary key(ord_num)
);
```

3 PROPOSED SOFTWARE

To test the validity of our methodology, we have developed an application programmed in Python language. A demonstration of the different operations is the subject of the following paragraphs [7].

Firstly, the user must log into the PostgreSQL system through the frame connection for choosing the database to migrate towards Cassandra (Figure11).



Figure11: Data migration application

Next, the frame “List of databases” provides, to the user, a set of tables from the database to migrate them. In the end, the frame “data migration” is used to define the tables of our relational database to migrate and, if necessary, define tables obtained from the merging of other tables to replace the joins of the relational model. The first migration is done with the button “Migrate tables”. The second is done by the button “Migrate merged tables”. A text file containing the CQL code is generated and must be imported into the Cassandra software to use the data of the source database. an extract of the application's source code is presented in the appendix

4 Conclusion

In this paper, we have focused on the migration from a relational database to a NoSQL database through reverse engineering. During migration, the model target (i.e., column-oriented model) was automatically created from the relational schema. To make the process a reality, we have developed a Python application based on the proposed methodology. As a perspective, our software can be enriched with other features. For example: collect queries and integrate them into the design of the NoSQL database, support of all relational and NoSQL databases whatever the DBMS to which they belong (not only PostgreSQL and Cassandra as illustrated in this work).

5 References

1. Hanine M., Benderrag A., Boutkhoul O., ‘Data Migration Methodology from Relational to NoSQL Databases’. 2015, World Academy of Science, Engineering and Technology International Journal of Computer, Electrical, Automation, Control and Information Engineering, Vol:9, No:12
2. Chebotko A., Kashlev A., Lu S., ‘A Big Data Modeling Methodology for Apache Cassandra’. 2015, IEEE International Congress on Big Data
3. https://docs.datastax.com/en/dse/6.0/cql/cql/cql_reference/cqlReferenceTOC.html (consulted on January 2021)
4. http://perso.univlemans.fr/~cpiau/BD/SQL_PAGES/SQL0.html#L5 (consulted on January 2021)
5. Abdelhedi F., Ait Brahim A., Atigui F., Zurfluh G., ‘MDA transformation process from a conceptual data schema to a logical NoSQL schema’. 2016, IT Congress of Organizations and Information and Decision-Making Systems (INFORSID 2016), Grenoble, French
6. Mason, R. T., ‘NoSQL databases and data modeling techniques for a document-oriented NoSQL database’. 2015, Proceedings of Informing Science & IT Education Conference (InSITE)
7. Giridhara Gopalan M., Prasanna C., Srihari Krishna YV., Shanthini B., Arulkumar A., ‘MYSQL TO CASSANDRA CONVERSION ENGINE’. 2017, IEEE 3rd International Conference on Sensing, Signal Processing and Security (ICSSS)
8. DOURHRI, A., HANINE, M., & OUAHMANE, H. . (2021). Methodology for modeling a column-oriented database with Cassandra. International Journal of Computer Engineering and Data Science (IJCEDS), 1(1), 14–21. Retrieved from <http://www.ijceds.com/ijceds/article/view/16>

6 Appendix

```

import tkinter as tk
from tkinter import ttk
fen=tk.Tk()
fen.title("Data Migration From RDBMS To NoSQL")

H,W=fen.winfo_screenheight(),fen.winfo_screenwidth()
#l,h=550,750;x,y=(W-1)//2,(H-h)//2;
geo="{x{}}+{{}}+{{}}".format(W,H,-10,0);fen.geometry(geo)
fen.geometry(geo)

cnx=tk.LabelFrame(fen,text="Connec-
tion",width=510,height=200,borderwidth=5,back-
ground="coral",foreground="black",font = ("Times New Ro-
man",14,"bold"))
content=tk.LabelFrame(fen,text="List of Data-
bases",width=510,height=400,borderwidth=5)
result=tk.LabelFrame(fen,text="Data migra-
tion",width=650,height=610,borderwidth=5)

cnx.place(x=10,y=10);content.place(x=10,y=220);re-
sult.place(x=550,y=10)

ces=tk.StringVar();cnx_lab_sys=tk.Label(cnx,text="RDBMS
(Source)",font = ("Times New Roman",12), background
="coral");

cnx_entry_sys=tk.ttk.Combobox(cnx,values=["Post-
GreSQL","MySQL","Oracle","Access"],textvariable=ces,back-
ground = 'black', foreground="blue",font = ("Times New
Roman",12),width=18)
cnx_entry_sys.current(0)
cnx_lab_sys.place(x=20,y=20);cnx_en-
try_sys.place(x=140,y=20)

ceu=tk.StringVar();cnx_lab_user=tk.La-
bel(cnx,text="User",font = ("Times New Roman",12), back-
ground = "coral");
cnx_entry_user=tk.Entry(cnx,width=20,textvaria-
ble=ceu,font = ("Times New Roman",12), foreground
="blue")
cnx_lab_user.place(x=20,y=65);cnx_en-
try_user.place(x=140,y=65);

```

```

cempdp=tk.StringVar();cnx_lab_mdp=tk.Label(cnx,text="Pass-
word",font = ("Times New Roman",12), background
="coral");
cnx_entry_mdp=tk.Entry(cnx,width=20,textvaria-
ble=cempdp,show="*",font = ("Times New Roman",12), fore-
ground = "blue")
cnx_lab_mdp.place(x=20,y=110);cnx_en-
try_mdp.place(x=140,y=110);

cnx_data_source=lambda :5
cnx_button=tk.Button(cnx,text="Connect to \ndata
source",command=cnx_data_source,width=15,height=3,font =
("Times New Roman",14))
cnx_button.place(x=330,y=35)

treeview=ttk.Treeview(content,height=17)

treeview["columns"]=("one","two","three")
treeview.column("#0", width=170, minwidth=170,
stretch=tk.NO)
treeview.column("one", width=100, minwidth=100,
stretch=tk.NO)
treeview.column("two", width=100, minwidth=100)
treeview.column("three", width=100, minwidth=100,
stretch=tk.NO)

scroll = ttk.Scrollbar(treeview, orient="vertical", com-
mand=treeview.yview)
scroll.place(x=5+447+2, y=2, height=343+20)
treeview.configure(yscrollcommand=scroll.set)

treeview.place(x=5,y=5)

treeview.insert('', '0', 'item1',text='databases')
treeview.insert('', '1', 'item2',text='BankDb')
treeview.insert('', 'end', 'item3',text='SchoolDb')

treeview.insert('item2', 'end', 'A',text='Account')
treeview.insert('item2', 'end', 'B',text='Operation')
treeview.insert('item2', 'end', 'C',text='Client')

treeview.insert('A', 'end', 'A1',text='Comptel')

```



```

treeview.insert('A','end','A2',text='Compte2')
treeview.insert('A','end','A3',text='Compte3')

treeview.insert('B','end','OP1',text='Operation1')
treeview.insert('B','end','OP2',text='Operation2')
treeview.insert('B','end','OP3',text='Operation3')

treeview.insert('C','end','Cl1',text='Client1')
treeview.insert('C','end','Cl2',text='Client2')
treeview.insert('C','end','Cl3',text='Client3')

treeview.insert('item3','end','D',text='Student')
treeview.insert('item3','end','E',text='Classe')
treeview.insert('item3','end','F',text='Teacher')

treeview.move('item2','item1','end')
treeview.move('item3','item1','end')

datam_lab_list_of_tab=tk.Label(result,text="RDBMS
(Source)",font = ("Times New Roman",12,"bold"), fore-
ground="white",background ="red",width=68)
datam_lab_list_of_tab.place(x=10,y=10)

tv=ttk.Treeview(result,columns=(1,2,3,4,5),show="head-
ings",height="5")
tv.place(x=10,y=55)
tv.heading(1,text="Name")
tv.column(1,width=120)
tv.heading(2,text="Schema")
tv.column(2,width=200)
tv.heading(3,text="Size (GB) ")
tv.column(3,width=65)
tv.heading(4,text="Rows")
tv.column(4,width=150)
tv.heading(5,text="Migration")
tv.column(5,width=80)

list_tables=["Account","Operation","Client"]
list_tables_sql=["create1","create2","create3"]
list_tables_size=[12.66,7833.55,1]
list_tables_rows=[12.66*10**9*1/40,7833.55*10**9*1/60,1*10**9*1/
50]

```

```

for i in range(3):
    tv.in-
sert("",tk.END,values=(list_tables[i],list_tables_sql[i],
list_tables_size[i],list_tables_rows[i],""))

migrate_datas=lambda :5
migrate_button=tk.Button(result,text="Migrate ta-
bles",command=migrate_datas,width=15,height=1,font =
("Times New Roman",14,"bold"),background="orange",fore-
ground="white")
migrate_button.place(x=10,y=200)

datam_lab_mergeTab=tk.Label(result,text="Tables to
merge",font = ("Times New Roman",14,"bold"), fore-
ground="white",background ="red",width=56)
datam_lab_mergeTab.place(x=10,y=300)

tv2=ttk.Treeview(result,columns=(1,2,3,4,5),show="head-
ings",height="5")
tv2.place(x=10,y=360)
tv2.heading(1,text="Table1")
tv2.column(1,width=125)
tv2.heading(2,text="Operation")
tv2.column(2,width=125)
tv2.heading(3,text="Table1")
tv2.column(3,width=125)
tv2.heading(4,text="Option")
tv2.column(4,width=135)
tv2.heading(5,text="Delete Merging")
tv2.column(5,width=110)

migrateMerged_button=tk.Button(result,text="Migrate
merged tables",command=mi-
grate_datas,width=17,height=1,font = ("Times New Ro-
man",14,"bold"),background="orange",foreground="white")
migrateMerged_button.place(x=10,y=500)

row1col1="Client"
row1col2=tk.Button(result,text="Merge with",command=mi-
grate_datas,width=17,height=1,font = ("Times New Ro-
man",14,"bold"),background="grey",foreground="black")
row1col3="Account"

```

18

```
row1col4="RadioButton"

tv2.insert("",tk.END,values=(row1col1,"",row1col3,"",""))

row1col2=tk.Button(result,text="Merge with",command=migrate_datas,width=10,height=1,font = ("Times New Roman",10,"bold"),background="grey",foreground="white")
row1col2.place(x=150,y=385)

row1col5=tk.Button(result,text="DELETE",command=migrate_datas,width=10,height=1,font = ("Times New Roman",10,"bold"),background="grey",foreground="white")
row1col5.place(x=535,y=385)

sel=lambda :5
var = tk.IntVar()
R1 = tk.Radiobutton(result, text="Client in Account",
variable=var, value=1,command=sel)
R1.place(x=395,y=385)
R2 = tk.Radiobutton(result, text="Account in Client",
variable=var, value=2,command=sel)
R2.place(x=395,y=410)
R3 = tk.Radiobutton(result, text="Detaillied Account",
variable=var, value=3,command=sel)
R3.place(x=395,y=435)

CheckVar1 = tk.IntVar()
CheckVar2 = tk.IntVar()
CheckVar3 = tk.IntVar()
C1 = tk.Checkbutton(result, text = "Yes", variable =
CheckVar1,onvalue = 1, offvalue = 0, height=1,width = 5)
C2 = tk.Checkbutton(result, text = "Yes", variable =
CheckVar2,onvalue = 1, offvalue = 0, height=1,width = 5)
C3 = tk.Checkbutton(result, text = "Yes", variable =
CheckVar3,onvalue = 1, offvalue = 0, height=1,width = 5)
C1.place(x=550,y=80)
C2.place(x=550,y=100)
C3.place(x=550,y=120)

fen.mainloop()
```